TradeX B.Tech Capstone — Technical Defense Guide

Person 3: Persistence, Portfolio Ledger & Quantitative Simulation Architect

CANDIDATE / DEFENDER: Person 3 **PRESENTATION TIME:** 5 to 7 Minutes

ENGINEERING ROLE: Persistence, Portfolio Ledger & Quantitative Simulation Architect

CORE DOMAIN: MySQL 8.0 Connection Pooling, Transaction Safety (ACID), Portfolio P&L; Math, Virtual Margin Wallet, Autonomous Market Simulator (Geometric Brownian Motion), Statutory Brokerage Calculation

ASSIGNED REPOSITORY FILES:
- **backend/include/database/Database.hpp**
- **backend/include/services/PortfolioService.hpp**
- **backend/include/services/WalletService.hpp**
- **backend/include/services/PnLService.hpp**
- **backend/include/services/WatchlistService.hpp**
- **backend/include/services/MarketSimulator.hpp**
- **database/schema.sql**
- **database/indexes.sql**
- **database/seed.sql**

1. Executive Summary & Architectural Ownership

Person 3 manages the financial state, mathematical models, and persistent storage of TradeX. This engineer is responsible for the thread-safe native MySQL connection pool, relational schema integrity across 12 tables, ACID transaction safety, the mathematical calculation of realized/unrealized portfolio P&L;, virtual margin wallet accounting, and the background market simulator implementing stochastic Geometric Brownian Motion (GBM) to drive live stock tick feeds.

Core Defense Pitch: *"I engineered the persistence layer, financial accounting models, and quantitative simulation engine. I built a thread-safe MySQL connection pool using libmysql, ensuring zero connection churn during heavy concurrency. My portfolio engine dynamically computes mark-to-market unrealized P&L; and weighted-average acquisition costs, while my autonomous market simulation worker uses Geometric Brownian Motion with Gaussian drift to simulate realistic Indian market volatility and stream live price ticks to the WebSocket transport layer."*

2. Assigned Files & Detailed Code Responsibilities

backend/include/database/Database.hpp (Thread-Safe MySQL Connection Pool):
- Implements Database singleton managing a thread-safe queue: std::queue<MYSQL*> connectionPool_.
- acquireConnection() & releaseConnection(): Synchronized via std::mutex poolMutex_ with auto-ping healthcheck (mysql_ping).
- query(): Executes SQL statements and returns strongly-typed row dictionaries (QueryResult).
- Transaction Management: Exposes beginTransaction(), commit(), and rollback() to prevent partial trade settlements.

backend/include/services/PortfolioService.hpp (Portfolio & P&L Engine):
- getCompletePortfolio(userId): Consolidates delivery holdings (CNC), intraday positions (MIS), wallet balance, and overall summary.
- Calculates **Weighted Average Price** upon adding shares:

$$AvgPrice_{new} = (Qty_{old} \times AvgPrice_{old} + Qty_{new} \times Price_{new}) / Qty_{total}$$

- Dynamically computes **Unrealized P&L**: (CurrentPrice - AvgPrice) × Quantity using live market prices.
- Computes overall portfolio gain percentage and today's day gain.

backend/include/services/WalletService.hpp (Virtual Margin & Cash Ledger):
• Manages virtual capital: availableBalance, investedAmount, and withdrawableAmount.
• deposit() & withdraw(): Atomic balance adjustments with overdraft protection.
• Financial Audit Ledger: Automatically appends immutable records to the transactions table on every cash event or order settlement.

backend/include/services/PnLService.hpp (Statutory Brokerage & Regulatory Fees):
• Implements realistic Indian financial statutory charges rules:
  • **Brokerage:** Flat ■20 or 0.03% (whichever is lower)
  • **Securities Transaction Tax (STT):** 0.1% on delivery (CNC)
  • **Exchange Transaction Fee:** 0.00345% of total turnover
  • **GST:** 18% on (Brokerage + Exchange Fee)
  • **Stamp Duty:** 0.015% on buy orders only.

backend/include/services/MarketSimulator.hpp (Geometric Brownian Motion Engine):
• Spawns dedicated background thread runLoop() executing every 1.5 to 2.0 seconds.
• Implements stochastic differential equation: $\Delta S = S \times (\mu \Delta t + \sigma \varepsilon \sqrt{\Delta t})$ using std::normal_distribution.
• Evaluates simulated Indian exchange hours (09:15 to 15:30 IST) with 24/7 demo override.
• Emits MarketTick structs containing updated LTP, day high, day low, volume, and percentage change.

database/schema.sql, indexes.sql, seed.sql (MySQL Relational Architecture):
• 12 relational tables with foreign keys and cascade rules.
• B-Tree performance indexes on (stock_id, timestamp) and user_id.
• Seeds 32 top Indian equities, demo user accounts, starter wallets, and OHLCV history.

3. Data Structures & Algorithms Implemented by Person 3

Data Structure / Algorithm	Time Complexity	Space Complexity	Engineering Role in TradeX
Mutex-Synchronized Queue (Connection Pool)	Acquire: O(1) Release: O(1)	O(P) where P = Pool Size	Pre-allocates MySQL socket handles in a FIFO queue, eliminating connection handshake latency during concurrent order processing.
Geometric Brownian Motion (GBM)	O(S) per tick cycle (S = 32 stocks)	O(1) Auxiliary	Simulates continuous-time stochastic equity price fluctuations using Box-Muller normal distribution random sampling.
B-Tree Clustered & Secondary Indexes	O(log N) lookup	O(N) index pages	Accelerates historical OHLCV chart retrieval and user order lookups; prevents database table scans.
Weighted-Average Cost Accumulator	O(1) per execution	O(1) Auxiliary	Maintains tax-compliant acquisition costs for equity holdings across fragmented buy orders.

4. How Person 3's Backend Connects to the Frontend

• **Portfolio Dashboard & Tables:** frontend/src/components/PortfolioSummary.tsx, HoldingsTable.tsx, and PositionsTable.tsx call portfolioApi.getPortfolio(). PortfolioService computes live valuation and returns JSON consumed by React.
• **Virtual Wallet & Transactions:** frontend/src/pages/Funds.tsx and Transactions.tsx interact with walletApi.deposit(), walletApi.withdraw(), and walletApi.getTransactions().
• **Real-Time Tick Streaming:** MarketSimulator::runLoop computes price updates and triggers HttpServer::broadcastWebSocket. The browser's websocket.ts captures these messages, animating the flashing green/red tickers across MarketTicker.tsx and Markets.tsx.

5. B.Tech Defense & Viva Voce Q&A; (Ready to Speak)

Q1: Why use a Database Connection Pool instead of opening a connection per HTTP request?
Answer: *"Establishing a TCP connection to MySQL involves a 3-way handshake, SSL negotiation, and database authentication, which takes 15 to 40 milliseconds per request. In an exchange processing concurrent orders, connection churn would exhaust server file descriptors. My connection pool pre-allocates 4 persistent connection pointers in a mutex-locked queue, allowing worker threads to acquire and release connections in O(1) time with sub-millisecond overhead."*

Q2: How does your Geometric Brownian Motion (GBM) mathematical model work?
Answer: *"Geometric Brownian Motion is the standard mathematical model for stock prices under the Black-Scholes framework. It consists of two components: a deterministic drift term representing expected return ($\mu \Delta t$), and a stochastic diffusion term representing market volatility ($\sigma \varepsilon \sqrt{\Delta t}$), where ε is a standard normal random variable generated via std::normal_distribution. This prevents stock prices from ever turning negative while generating realistic random-walk price paths."*

Q3: How do you guarantee data consistency and prevent race conditions when two orders execute concurrently?
Answer: *"Order execution involves multiple state transitions: deducting wallet funds, creating order records, updating holdings, and appending to transaction logs. I wrap these multi-table operations within MySQL ACID transactions (BEGIN ... COMMIT / ROLLBACK) combined with C++ mutex locks. If any step fails—such as a user having insufficient margin—the entire transaction rolls back, guaranteeing zero negative balances and 100% financial consistency."*